Requirements Gathering & Specification Document

Stakeholder Analysis, Use Cases & User Stories, Functional & Non-Functional Requirements

Project Field	Project Specification Details
Project Title	EduHub — Full-Stack Online Learning Platform
Initiative & Program	Digital Egypt Pioneers Initiative (DEPI) — Software Engineering Track
Document Ref / Code	EDU-DOC-03 / DEPI-MS1
Document Version / State	Version 1.0 (Official Milestone 1 Submission)
Submission Due Date	16 October 2026
Architecture & Stack	N-Tier Architecture | ASP.NET Core MVC, EF Core, SQL Server, Azure, Stripe
Project Team Members	1. Aya Abdelmoniem Mohamed Hamed 2. Ibrahim Mohamed Ibrahim Abdo 3. Reem Ashraf Mohammed Ahmed 4. Ahmed Ramadan Ali Mohamed 5. Warda Mahmoud Khalaf Ahmed
Team Operating Model	Cross-Functional Collaborative Team (Collective ownership across all components; all members actively participate in full-stack implementation, architecture, DB, and QA)

1. Stakeholder Analysis — Key Stakeholders & Their Needs

A comprehensive Stakeholder Analysis was conducted to identify every internal actor, external user group, governance body, and third-party integration partner interacting with EduHub, capturing their core needs, pain points, influence levels, and success criteria.

1.1 Stakeholder Analysis Matrix

Stakeholder / Actor	Category & Influence	Key Needs & Expectations	Pain Points Solved by EduHub	Success Criteria
Students (Learners)	Primary End-User (High Interest / High Impact)	• Easy course discovery & filtering • Affordable or free structured courses • Smooth video playback & resume • Clear progress tracking & quizzes • Verifiable completion certificates	Eliminates scattered learning materials, unverified payment pages, lost video playback positions, and unaccredited course completion.	Can browse, enroll/pay, stream lessons, pass quizzes, and download PDF certificates seamlessly.
Instructors (Educators)	Primary Content Creator (High Interest / High Impact)	• Intuitive curriculum builder (Sections & Lessons) • Secure video hosting against piracy • Flexible pricing & discount coupons • Transparent enrollment & revenue stats	Prevents unauthorized video downloading via private Azure Blobs + SAS URLs and provides flexible promotional pricing.	Can author courses, upload media, configure coupons, publish content, and monitor student enrollments.
Administrators (Platform Governance)	Internal Governance (High Authority / High Impact)	• Centralized user & role control • Category & subcategory management • Course review & moderation workflow • Platform health & issue monitoring	Prevents low-quality or unverified courses from going live without review and enforces role security.	Full administrative control over users, categories, course approvals, and platform settings.
DEPI Evaluators & Project Mentors	Academic & Technical Sponsors (High Authority)	• Clean N-Tier software architecture • Proper design patterns (Repo, UoW, Strategy, Observer) • On-time milestone documentation & working MVP	Demonstrates enterprise-ready software engineering standards rather than ad-hoc monolithic code.	All 4 DEPI milestones delivered on schedule with clean code, tests, and full documentation.

Project Engineering Team (5 Members)	Development & QA Team (High Interest / High Impact)	• Clear API & DB contracts • Modular decoupled layers for parallel full-stack work • Coordinated Git/GitHub workflow	Avoids merge conflicts and architectural bottlenecks through N-Tier separation and Collective Ownership.	All 5 members contribute across Web, Services, Data, and Tests with zero blocking integration defects.
External Systems (Stripe & Azure Blob)	Technical Integration Partners (System Actors)	• Validated API requests & signed webhooks • Secure token-based media requests • Idempotent event handling	Offloads PCI-DSS payment card handling to Stripe and heavy media storage to Azure cloud infrastructure.	100% verified webhook processing and 15-minute time-limited SAS media delivery.

1.2 Role-Based Authorization Matrix

To protect stakeholder interests, EduHub enforces both role-based and resource-level ownership permissions across all platform capabilities:

Platform Functionality	Student Role	Instructor Role	Admin Role
Browse & search published courses	Yes	Yes	Yes
Enroll in free or paid courses	Yes	As student (if supported)	As student (if supported)
Watch enrolled course video content	Yes (Enrolled only)	Own courses or if enrolled	Administrative review access
Create new courses & curriculum	No	Yes	According to policy
Edit or archive courses	No	Yes (Own courses only)	Administrative moderation
Manage all users & role assignments	No	No	Yes
Manage categories & subcategories	No	No	Yes
Publish or moderate submitted courses	No	Submit for review / publish	Yes (Approve / Reject)
Track personal learning progress & quizzes	Yes	As student (if supported)	Authorized admin view
View instructor enrollment & revenue stats	No	Yes (Own earnings only)	Authorized financial admin
Receive / Issue PDF Certificates	Eligible students receive	No direct manual issuance	Admin corrections if permitted

2. User Stories & Use Cases — System Interaction Scenarios

This section illustrates how actors interact with EduHub through a high-level System Use Case Diagram, detailed Use Case Specifications (Preconditions, Step-by-Step Main Flows, Alternative/Exception Flows, and Postconditions), and Agile User Stories.

2.1 System Use Case Diagram

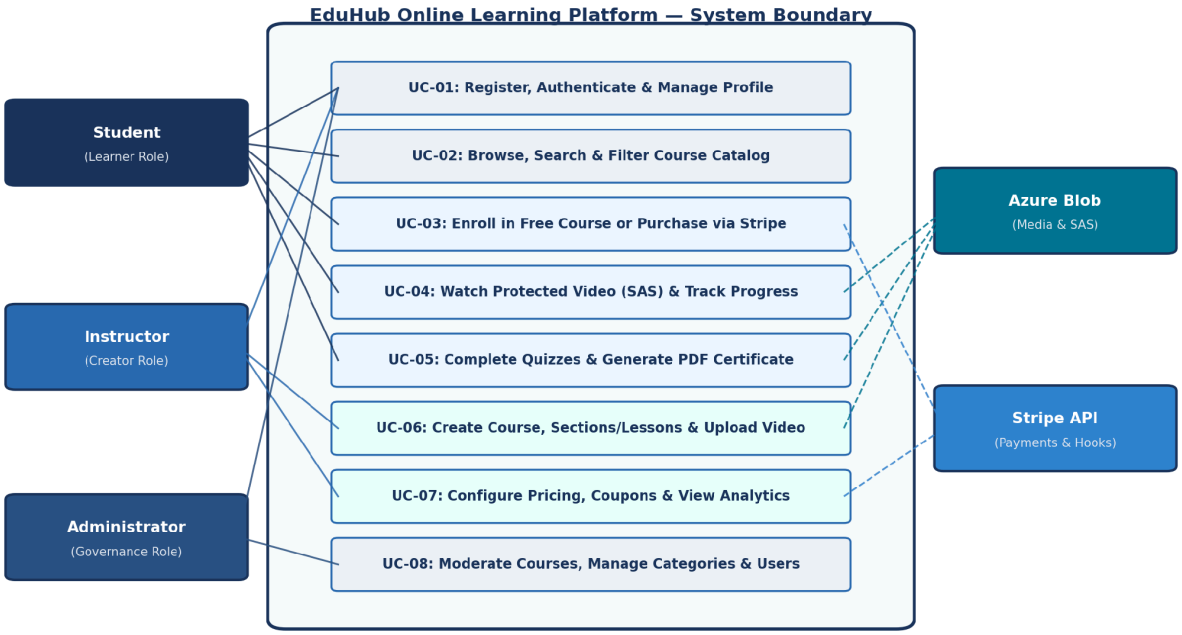


Figure 1: EduHub System Use Case Diagram & External Actor Interactions

2.2 Detailed Use Case Specifications (End-to-End Interaction Scenarios)

Use Case ID & Name	Actors & Preconditions	Main Success Scenario (Step-by-Step Flow)	Alternative / Exception Flows & Postconditions
UC-01: Free Course Enrollment & Learning Workflow (Scenario A)	Actor: Student Preconditions: • User is registered & logged in as Student. • Course status is 'Published' with Price = $0.00.	1. Student searches/filters catalog and views course details. 2. Student clicks 'Enroll Now (Free)'. 3. System validates no duplicate enrollment exists and creates `Enrollment` record. 4. Student opens course player; system generates short-lived Azure SAS URL. 5. Student watches lessons via Video.js; system saves playback progress & lesson completion. 6. Student completes required quiz and qualifies for certificate.	Exception E1: Student already enrolled -> Redirect directly to learning dashboard. Postcondition: Active enrollment created; lesson progress persisted on server.

UC-02: Paid Course Purchase via Stripe & Coupons (Scenario B)	Actor: Student, Stripe API Preconditions: • Student is authenticated. • Course is 'Published' with Price > $0.	1. Student selects paid course and optionally enters a coupon code. 2. Server validates coupon via Strategy Pattern (`IPricingStrategy`) and calculates final price. 3. System creates `Payment` record (Status: Pending) and initiates Stripe Checkout Session. 4. Student completes card payment on Stripe hosted page. 5. Stripe sends signed webhook event (`checkout.session.completed`) to EduHub backend. 6. Server verifies webhook signature & idempotency, marks payment 'Paid', and creates `Enrollment`.	Exception E1: Expired/invalid coupon -> Reject discount & show validation error. Exception E2: Browser redirects to success URL before webhook -> Do NOT grant access until verified webhook arrives. Postcondition: Verified payment recorded; student granted course entitlement.
UC-03: Instructor Course Authoring & Video Upload (Scenario C)	Actor: Instructor, Azure Blob Storage Preconditions: • User is authenticated with 'Instructor' role.	1. Instructor opens Dashboard and creates a new course (Status: 'Draft'). 2. Instructor configures title, category, level, price, and uploads thumbnail. 3. Instructor creates Sections and ordered Lessons (`SortOrder`). 4. Instructor selects MP4 video for a lesson; backend validates ownership, file type, and size. 5. Video is uploaded to private Azure Blob container; storage reference & duration saved in DB. 6. Instructor submits course for publication; system validates required fields and transitions status to 'PendingReview' or 'Published'.	Exception E1: Instructor attempts to edit another instructor's course ID -> Reject with HTTP 403 Forbidden. Exception E2: Missing sections/videos on publish -> Block transition & list validation errors. Postcondition: Course structure and private media references stored consistently.
UC-04: Quiz Assessment & Automated PDF Certificate	Actor: Student Preconditions: • Student is enrolled in course. • Required lessons are completed.	1. Student launches section quiz; server sends questions/options without exposing `IsCorrect` flags. 2. Student submits answers within allowed attempts. 3. Server grades attempt, records `QuizAttempt` score, and marks `Passed = true`. 4. Observer event triggers completion check: verifies 100% lessons completed + required quizzes passed. 5. System confirms no certificate exists yet, generates PDF via iTextSharp with unique `CertificateNumber`, stores reference in Azure Blob, and enables student download.	Exception E1: Score below `PassingScore` -> Record failed attempt; allow retry if `AttemptsAllowed` permits. Exception E2: Retried completion event -> Idempotent check prevents duplicate certificates. Postcondition: Unique verifiable PDF certificate issued and stored.

UC-05: Admin Moderation & Security Enforcement (Scenario D)	Actor: Administrator Preconditions: • User authenticated with 'Admin' role.	1. Admin opens Moderation Dashboard and views courses in 'PendingReview' status. 2. Admin previews course metadata, sections, and lesson videos. 3. Admin approves course (Status -> 'Published') or rejects with feedback (Status -> 'Rejected'). 4. Admin manages category hierarchy and user role assignments.	Exception E1: Unauthorized user attempts admin or unowned resource route -> System blocks request and logs security event. Postcondition: Platform catalog quality and RBAC rules enforced.

2.3 Agile User Stories & Acceptance Criteria

Story ID & Role	User Story Statement (As a... I want... So that...)	Acceptance Criteria (Given / When / Then)
US-01 (Student)	As a student, I want to search and filter courses by category, level, price, and rating so that I can quickly find relevant courses.	Given published courses exist in the database, When I search by keyword or filter by category/level/price, Then only matching published courses are displayed with accurate details.
US-02 (Student)	As a student, I want to enroll in free courses immediately and purchase paid courses securely via Stripe so that I can start learning.	Given I select a paid course, When I apply a valid coupon and complete Stripe Checkout, Then enrollment is activated only after server-side webhook verification.
US-03 (Student)	As an enrolled student, I want to watch lessons via Video.js, resume from my last playback position, and track my completion percentage.	Given I am enrolled in a course, When I watch a lesson and mark it completed, Then my server-side `LessonProgress` updates and my overall course progress bar recalculates.
US-04 (Student)	As an enrolled student, I want to bookmark lessons, take private notes, and submit course ratings/reviews.	Given I am watching an enrolled lesson, When I save a note or submit a 1–5 star course review, Then the system validates my enrollment and persists my data.
US-05 (Student)	As an eligible student, I want to pass course quizzes and download a verifiable PDF certificate upon finishing the course.	Given I completed all required lessons and passed required quizzes, When completion is evaluated, Then a unique PDF certificate is generated once and made downloadable.
US-06 (Instructor)	As an instructor, I want to create courses, reorder sections and lessons, and save drafts before publishing.	Given I am logged in as an Instructor, When I create or reorder sections/lessons on my own course, Then the sort order and draft state are saved and protected from other instructors.
US-07 (Instructor)	As an instructor, I want to upload course thumbnails and lesson videos to Azure Blob Storage with secure access.	Given I upload a lesson video, When the backend validates file type, size, and course ownership, Then the video is stored in Azure Blob Storage and served only via short-lived SAS URLs.
US-08 (Instructor)	As an instructor, I want to configure course prices, promotional discounts, view enrollment/revenue stats, and post announcements.	Given I own published courses, When I open my Instructor Dashboard, Then I can view accurate student enrollment counts, revenue summaries, and publish course announcements.

US-09 (Admin)	As an administrator, I want to manage users, roles, categories, and moderate submitted courses so that platform quality is maintained.	Given I am logged in as Admin, When I manage categories, assign roles, or review a pending course, Then changes apply platform-wide with audit integrity.

3. Functional Requirements — Complete Feature Specifications

The functional requirements catalog specifies every feature and business rule required across EduHub's 10 functional modules:

Module / Req ID	Feature Name	Functional Specification & Business Rules	Priority
FR-AUTH-01 to 05 Identity & Roles	Authentication & RBAC	• Register, log in, log out, and recover accounts via ASP.NET Core Identity. • Support `Student`, `Instructor`, and `Admin` roles plus `StudentProfile` and `InstructorProfile`. • Enforce resource-level ownership (instructors can only edit their own courses).	Must Have (MVP)
FR-CAT-01 to 06 Course Discovery	Catalog, Search & Filtering	• Browse published courses with thumbnail, title, price, rating, and instructor info. • Search by title/keywords and filter by category, subcategory, level, price, or rating. • View course curriculum sections and watch designated preview lessons (`IsPreview = true`).	Must Have (MVP)
FR-CRS-01 to 07 Course Authoring	Instructor Dashboard & Builder	• Create, edit, and archive courses with statuses: `Draft`, `PendingReview`, `Published`, `Rejected`, `Archived`. • Create and reorder `CourseSection` and `Lesson` entities (`SortOrder`). • Validate required fields before publishing; display enrollment and revenue statistics.	Must Have (MVP)
FR-MED-01 to 05 Video & Media	Azure Blob Upload & Delivery	• Validate ownership, file types, and sizes on instructor video/thumbnail upload. • Store videos in private Azure Blob Storage and save metadata (`DurationSeconds`, `StorageReference`). • Deliver protected videos to enrolled students via short-lived signed SAS URLs and Video.js.	Must Have (MVP)
FR-ENR-01 to 06 Enrollment & Progress	Enrollment & Lesson Tracking	• Support immediate free-course enrollment and verified paid-course enrollment. • Prevent duplicate active enrollments for the same `(StudentId, CourseId)`. • Track server-side `LessonProgress` (`LastPositionSeconds`, `IsCompleted`, `CompletedAt`) and completion percentage.	Must Have (MVP)

FR-PAY-01 to 07 Payments & Pricing	Stripe & Strategy Pattern Coupons	• Calculate discounts on the server using Strategy Pattern (Regular, Percentage, Fixed-amount, Coupons). • Validate coupon eligibility, `StartDate`, `ExpirationDate`, `UsageLimit`, and `CouponRedemption`. • Process payments via Stripe (Test Mode), verify webhooks, enforce idempotency, and record `Payment` states.	Must Have (MVP)
FR-QZ-01 to 06 Assessments	Quizzes & Auto-Grading	• Attach `Quiz` entities to lessons/sections with `PassingScore` and `AttemptsAllowed`. • Manage `QuizQuestion` and `QuizOption` without exposing `IsCorrect` to clients before grading. • Record `QuizAttempt` (`Score`, `Passed`, `StartedAt`, `CompletedAt`).	Must Have (MVP)
FR-CRT-01 to 05 Certificates	PDF Certificate Generation	• Verify completion rules (all required lessons completed + required quizzes passed). • Prevent duplicate certificate issuance under the same policy. • Generate downloadable PDF certificates via iTextSharp with unique `CertificateNumber` and store reference.	Must Have (MVP)
FR-INT-01 to 05 Interactive Features	Reviews, Notes, Bookmarks & Q&A	• Allow eligible enrolled students to submit `CourseReview` (Rating + Comment). • Support student `LessonBookmark` and private `LessonNote` records. • Support `CourseAnnouncement`, `CourseQuestion`, `CourseAnswer`, and `Notification`.	Should Have
FR-ADM-01 to 04 Administration	Platform Moderation & Settings	• Manage user accounts and role assignments. • Create and manage hierarchical `Category` and subcategory records. • Review, approve, or reject submitted courses and monitor platform activity.	Must Have (MVP)

4. Non-Functional Requirements — Quality & Operational Criteria

Non-functional requirements define the strict operational standards across Performance, Security, Usability, and Reliability/Maintainability:

Quality Pillar	Req ID & Attribute	Measurable Requirement & Technical Enforcement
1. Performance & Scalability	NFR-PERF-01: Web & API Latency	Server-rendered MVC views shall load within < 1.5 seconds and JSON endpoints within < 300 ms under normal concurrent load.
1. Performance & Scalability	NFR-PERF-02: Media Streaming & DB Indexing	Video playback via Video.js and Azure Blob Storage shall start within < 1.8s TTFB; SQL Server tables shall index

		foreign keys, search titles, and `(StudentId, CourseId)` lookups.
2. Security & Privacy	NFR-SEC-01: Auth & Resource Ownership	Enforce ASP.NET Core Identity authentication, role policies, and strict resource-level ownership checks (preventing IDOR attacks on courses, progress, and certificates).
2. Security & Privacy	NFR-SEC-02: Protected Media & Payment Integrity	Never expose permanent public URLs for paid videos (use short-lived SAS URLs); never trust client-side prices; verify Stripe webhook signatures on the server over HTTPS.
2. Security & Privacy	NFR-SEC-03: Web Vulnerability & Secret Protection	Mitigate OWASP Top 10 (CSRF, XSS, SQLi); store only necessary payment references; exclude secrets, passwords, and connection strings from source control and logs.
3. Usability & Accessibility	NFR-USA-01: Responsive Bootstrap 5 Design	Provide a consistent, responsive UI across desktop, tablet, and mobile viewports using Bootstrap 5 and clear visual progress indicators (`Progress.js`).
3. Usability & Accessibility	NFR-USA-02: User Feedback & Accessibility	Provide clear validation messages, intuitive navigation across categories/lessons, and WCAG 2.1 AA contrast and keyboard accessibility.
4. Reliability & Maintainability	NFR-REL-01: Transaction Integrity & Idempotency	Coordinate related database writes via Unit of Work transactions; ensure webhook handlers and Observer event handlers (certificates, notifications, enrollments) are idempotent on retry.
4. Reliability & Maintainability	NFR-REL-02: Modular N-Tier Architecture	Maintain strict dependency flow (`EduHub.Web -> EduHub.Services -> EduHub.Data`) with Dependency Injection and encapsulated external service interfaces for testability.

5. Requirements Traceability Matrix (RTM)

The traceability matrix links Stakeholder Needs, Use Cases, Functional Requirements, Architecture Components, and Verification Scenarios:

Stakeholder Need	Use Case / Story	Req ID	N-Tier Layer & Pattern	Verification Test
Role-based secure access	UC-01, UC-05	FR-AUTH-01..05 NFR-SEC-01	EduHub.Web / Identity	Integration & Security Tests
Course discovery & search	UC-01 / US-01	FR-CAT-01..06 NFR-PERF-01	EduHub.Services / CourseRepo	Unit & UI Filtering Tests
Instructor course authoring	UC-03 / US-06	FR-CRS-01..07	EduHub.Services / UnitOfWork	Unit Test (Publish Validation)
Protected video streaming	UC-03 / US-07	FR-MED-01..05 NFR-SEC-02	EduHub.Services / AzureBlob	Security Test (SAS Entitlement)
Progress & lesson tracking	UC-01 / US-03	FR-ENR-01..06	EduHub.Services / Observer	Integration Test (Progress Calc)
Paid checkout & coupons	UC-02 / US-02	FR-PAY-01..07 NFR-REL-01	EduHub.Services / Strategy	Unit (Pricing) & Webhook Test

Quiz grading & PDF certs	UC-04 / US-05	FR-QZ-01..06 FR-CRT-01..05	EduHub.Services / iTextSharp	E2E Scenario A & B Verification